# DL_LAB3

資工碩一

**ID : 314551147**

**Name : 楊承恩**

## 1. Introduction

This lab focuses on image inpainting, where the input is an image with missing regions, and the model must generate semantically consistent and visually natural content to fill them in.
The approach adopts VQGAN + MaskGIT: VQGAN encodes the image into the discrete token space of a codebook, while MaskGIT's bidirectional Transformer performs parallel predictions over the token sequence. Through MVTM, the inpainting process is carried out progressively from easy to hard.

Compared to autoregressive token-by-token generation, MaskGIT's iterative decoding can update multiple unknown tokens at each step, achieving a balance between speed and quality.

# 2. Implementation Details

## A. The details of your model (Multi-Head Self-Attention)

```python
class MultiHeadAttention(nn.Module):
    def __init__(self, dim=768, num_heads=16, attn_drop=0.1):
        super().__init__()
        self.dim = dim
        self.num_heads = num_heads
        self.head_dim = dim // num_heads
        self.qkv = nn.Linear(dim, dim * 3, bias=True)
        self.scale = 1.0 / math.sqrt(self.head_dim)
        self.attn_drop = nn.Dropout(attn_drop)
        self.proj = nn.Linear(dim, dim)

    def forward(self, x):
        B, N, C = x.shape

        qkv = self.qkv(x).reshape(B, N, 3, self.num_heads, self.head_dim)
        qkv = qkv.permute(2, 0, 3, 1, 4).contiguous()
        q, k, v = qkv[0], qkv[1], qkv[2]

        attn = torch.matmul(q, k.transpose(-2, -1)) * self.scale
        attn = attn.softmax(dim=-1)
        attn = self.attn_drop(attn)

        y = torch.matmul(attn, v)
        y = y.transpose(1, 2).contiguous()
        y = y.view(B, N, C)

        y = self.proj(y)
        return y
```

### Head dimension split

The input vector has a dimension of dim = 768. I set num_heads = 16, so each head has a dimension of 48.

### Building Q, K, V projections

I use a linear layer nn.Linear(dim, dim * 3) to generate the query, key, and value vectors simultaneously. The output size is three times dim since each token requires corresponding Q/K/V vectors.

### Scaling factor

When computing attention scores, if the values are too large, the softmax tends to become one-hot, causing gradients to vanish. Therefore, I scale the scores.

### Forward

First, I apply a linear transformation to the input x to obtain qkv, then reshape it into the form (B, N, num_heads, head_dim). Next, I use permute to adjust the dimensions and separate out q, k, and v.Then, I compute the attention weights attn, multiply them with v, and obtain the updated representation of each token after multi-head attention.

Finally, I concatenate the outputs of all heads back into the original dimension (B, N, C) and pass them through a linear layer to produce the final output.

## B. The details of your stage2 training (MVTM, forward, loss)

### 1. MVTM

In the Stage-2 MVTM training process, my model integrates VQGAN as the encoder and Transformer as the decoder. It includes three core functions: encode_to_z, gamma_func, and forward.

```
def encode_to_z(self, x: torch.Tensor):
    z_q, idx, _ = self.vqgan.encode(x)
    idx = idx.view(x.size(0), -1).contiguous().long()
    return z_q, idx
```

This function passes the input image x through the VQGAN encoder, converting it into a latent representation z_q, and simultaneously quantizes it into discrete token indices.

```
def gamma_func(self, mode: str = "cosine"):
    mm = (mode or "cosine").lower()
    if mm == "linear":
        return lambda r: 1.0 - float(r)
    elif mm == "cosine":
        return lambda r: float(np.cos(float(r) * np.pi * 0.5))
    elif mm == "square":
        return lambda r: float((1.0 - float(r)) * (1.0 - float(r)))
    else:
        return lambda r: 1.0 - float(r)
```

This function determines the scheduling strategy for the mask ratio. During training, the mask ratio is randomly sampled; during inference, the mask ratio is adjusted according to the proportion of the current iteration. It supports three different scheduling methods.

```
def forward(self, x: torch.Tensor):
    _, gt_idx = self.encode_to_z(x)

    r = np.random.rand()
    p = self.gamma(r)

    rnd  = torch.rand_like(gt_idx, dtype=torch.float32)
    msk  = rnd < p
    tok  = gt_idx.clone()
    tok[msk] = self.mask_token_id

    logits = self.transformer(tok)

    return logits, gt_idx
```

The input image is quantized into token indices through the VQGAN encoder.A random ratio is generated, and the mask ratio is determined using gamma_func.A mask is created, replacing

part of the tokens with mask_token_id.The masked tokens are fed into the Transformer for prediction, producing the logits.

## 2. Forward and Loss

In Stage-2 training, my model first converts the input image into latent tokens, and then uses a Transformer to predict the token probability distribution at each position. The core of the entire training process lies in the computation of Cross-Entropy Loss and backpropagation.

```
def train_one_epoch(self, train_loader, epoch):
    self.model.train()
    self.optim.zero_grad(set_to_none=True)

    total_loss = 0.0
    accum = max(1, self.args.accum_grad)

    pbar = tqdm(enumerate(train_loader), total=len(train_loader))
    for step, image in pbar:
        x = image.to(self.args.device, non_blocking=True)

        logits, z_indices = self.model(x)

        loss = F.cross_entropy(
            logits.reshape(-1, logits.size(-1)),
            z_indices.reshape(-1),
        )

        (loss / accum).backward()
        total_loss += loss.item()

        if (step + 1) % accum == 0 or (step + 1) == len(train_loader):
            self.optim.step()
            self.optim.zero_grad(set_to_none=True)

        pbar.set_description(f"(train) Epoch {epoch} | loss {loss.item():.4f}", refresh=False)

    if self.scheduler is not None:
        self.scheduler.step()

    avg_loss = total_loss / len(train_loader)
    if self.args.log and self.writer is not None:
        import wandb
        self.writer.add_scalar("Loss/train", avg_loss, epoch)
        wandb.log({"Loss/train": avg_loss, "epoch": epoch})
    return avg_loss
```

In train_one_epoch, each batch of data is first fed into the model, and then cross-entropy loss is used to measure the gap between the predictions and the ground truth. The reshape operation here flattens the 3D tensor into a 2D tensor.

During backpropagation, to avoid running out of GPU memory, I use gradient accumulation. Specifically, accum = max(1, self.args.accum_grad) determines how many steps to accumulate before performing one optimizer update. This allows training with a simulated large batch size even under limited GPU memory.

```python
def configure_optimizers(self):
    opt = torch.optim.Adam(self.model.parameters(),
                           lr=self.args.learning_rate,
                           betas=(0.9, 0.96))
    sch = None
    return opt, sch
```

In configure_optimizers, Adam is used to update the parameters. Here, the betas are set to (0.9, 0.96), which correspond to the smoothing parameters for the first- and second-order moment estimates, helping to achieve stable convergence.

## C. The details of your inference for inpainting task (iterative decoding)

In inference, I use the iterative decoding method to perform inpainting.

```python
def inpainting(self, image, mask_b, i):
    maska = torch.zeros(self.total_iter, 3, 16, 16)
    imga  = torch.zeros(self.total_iter + 1, 3, 64, 64)

    mean = torch.tensor([0.4868, 0.4341, 0.3844], device=self.device).view(3, 1, 1)
    std  = torch.tensor([0.2620, 0.2527, 0.2543], device=self.device).view(3, 1, 1)

    ori = (image[0] * std) + mean
    imga[0] = ori.detach().cpu()

    with torch.no_grad():
        _, z_idx0 = self.model.encode_to_z(image)
        z_work = z_idx0.clone().to(self.device)
        cur_mask = mask_b.to(self.device)

        last_vis = None
        for step in range(self.total_iter):

            if step == self.sweet_spot:
                break

            ratio = float(step + 1) / float(self.total_iter)

            z_work, cur_mask = self.model.inpainting(
                z_work, cur_mask, ratio, self.mask_func
            )

            m16 = cur_mask.view(1, 16, 16).float().cpu()

            m_img = torch.ones(3, 16, 16)
            pos = (m16[0] > 0.5).nonzero(as_tuple=False)
            if pos.numel() > 0:
                m_img[:, pos[:, 0], pos[:, 1]] = 0.0
            maska[step] = m_img


            shape = (1, 16, 16, 256)
            zq = self.model.vqgan.codebook.embedding(z_work.long()).view(shape)
            zq = zq.permute(0, 3, 1, 2).contiguous()
            out = self.model.vqgan.decode(zq)
            vis = (out[0] * std) + mean
            last_vis = vis
            imga[step + 1] = vis.detach().cpu()
```

First, I create two tensors. Then, I compute the normalized result of the input image and save the original input as the 0th result. Using encode_to_z, I convert the input image into latent space indices and prepare a cur_mask as the initial mask.

At each iteration, I calculate a ratio that determines the current mask update rate. I call the inpainting function from the second figure to update z_work and cur_mask, then convert the new mask into an image and store it in maska. The updated latent indices are passed into the VQGAN decoder to reconstruct an image, which is stored in imga.

```python
def inpainting(self,
               z_indices: torch.Tensor,
               mask_bc: torch.Tensor,
               ratio: float,
               mask_func: str = None):

    inp = z_indices.clone()
    inp[mask_bc] = self.mask_token_id

    logits = self.transformer(inp)
    probs  = torch.softmax(logits, dim=-1)

    pick_prob, pick_idx = probs.max(dim=-1)

    pick_prob = pick_prob.masked_fill(~mask_bc, float('inf'))


    g = -torch.empty_like(pick_prob).exponential_().log()
    temp = self.choice_temperature * (1.0 - float(ratio))
    confidence = pick_prob + temp * g

    new_tokens = torch.where(mask_bc, pick_idx.to(z_indices.dtype), z_indices)

    sched = self.gamma if mask_func is None else self.gamma_func(mask_func)
    keep_ratio = sched(ratio)

    cur_mask_cnt = mask_bc.sum(dim=1).to(torch.float32)
    keep_num     = (keep_ratio * cur_mask_cnt).floor().to(torch.long)

    _, order = torch.sort(confidence, dim=1, descending=False)

    next_mask = torch.zeros_like(mask_bc)

    B = order.size(0)
    for b in range(B):
        k = int(keep_num[b].item())
        if k > 0:
            idxs = order[b, :k]
            next_mask[b].scatter_(0, idxs, True)

    return new_tokens, next_mask
```

I first make a copy of the input z_indices and replace the positions specified by mask_bc with the mask token ID. The masked indices are then fed into the Transformer to obtain the predicted logits, which are passed through a softmax to get the probability distribution for each position. The token with the highest probability is selected as the prediction pick_idx, and the corresponding probability is recorded as pick_prob.

Next, Gumbel noise is added and multiplied by the temperature to obtain the final confidence score. The gamma_func is used to decide how many tokens to keep in the current iteration. Based on the confidence scores, the tokens are ranked, and the ones to be kept or re-masked are determined. Finally, new_tokens and next_mask are updated and returned for use in the outer loop of the first figure.

## 3. Discussion

I implemented three mask scheduling methods: linear, cosine, and square. As the results table shows, the FID score for three methods is very close, with the square method achieving the best score.It indicates that its generated results are closest to the real images. This may be because square scheduling releases more masks earlier in the process, allowing the model to access the full contextual information sooner and thereby produce more stable images.

Linear FID = 38.079

Cosine FID = 37.6541

Square FID = 37.3001

## 4. Experiment Score

### Part1: Prove your code implementation is correct

#### (a)Mask in latent domain

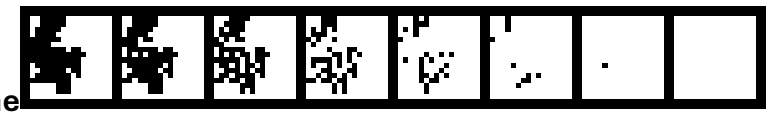
The masking conditions in the latent space after each iteration under different mask scheduling methods (cosine, linear, square).

**Cosine:** The mask ratio decreases more smoothly, with more masks retained in the early stage and gradually reduced in the later stage.

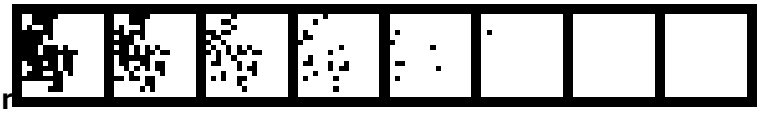
**Linear:** The number of masks decreases linearly, removing the same proportion of masks at each iteration in a regular manner.

**Square:** The mask ratio drops quickly at the beginning, but the decrease becomes slower in the later stage.

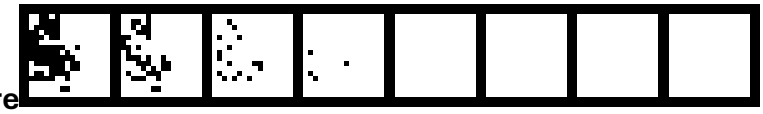
**Cosine**



**Linear**



**Square**



**(b)Predicted image**

The process of gradual image restoration under different mask scheduling methods.

**Cosine:** The reconstruction process is smoother, with the image gradually becoming clearer.

**Linear:** The convergence speed is faster, and a relatively complete image can be seen in the mid-stage.

**Square:** The early-stage generation performs very well, but the improvement becomes smaller in the later stage.

**Cosine**

**Linear**

**Square**

## Part2: The Best FID Score

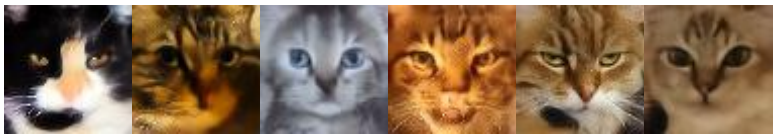
### Screenshot

```
(lab3) D:\DLP>python faster-pytorch-fid\fid_score_gpu.py --predicted-path .\test_results_square --gtcsv-path faster-p
ytorch-fid\test_gt.csv --device cuda:0 --num-workers 0
747
100%|                                                                                    15/15 [00:09<00:00,  1.62it/s]
100%|                                                                                    15/15 [00:02<00:00,  5.27it/s]
FID:  37.30012055300219
```

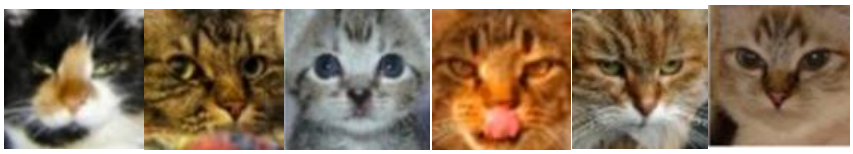
### Masked Images v.s MaskGIT Inpainting Results v.s Ground Truth

**Masked Images :**

**MaskGIT Inpainting Results :**



**Ground Truth :**



## The setting about training strategy, mask scheduling parameters, and so on

learning rate: 1e-4

batch size: 10

epochs: 50

optimizer: Adam

sweet spot: 8

total iteration: 8

mask function: square